Assignment #4

AI Content Generation Platform

Build a Full-Stack Content Factory Powered by LLMs

Organization:	Excellence Technologies Pvt Ltd
Phase:	Phase 1 - GenAI Fundamentals
Backend:	Django (Python)
Frontend:	React.js
Duration:	3 - 4 Days
Difficulty:	Intermediate to Advanced

Content Types Covered

Blog Posts | Social Media | Email Campaigns | Product Descriptions | Ad Copy
SEO Content | Newsletters | Landing Page Copy | Translations | Summaries

This is a complex assignment. Please read all sections carefully before beginning.

1. Assignment Overview

In this assignment, you will build an AI Content Generation Platform - a full-stack web application that acts as a content factory for marketing teams, bloggers, and businesses. Users can generate high-quality, structured content across multiple formats (blogs, social media, emails, product descriptions, ads) using LLMs, with fine-grained control over tone, style, audience, and SEO.

This is your first project using Django (Python) as the backend framework and React.js on the frontend. You will learn Django's powerful ORM, admin panel, REST framework, and how it differs from Flask and Express.js. The platform must feel like a real SaaS product - with workspaces, content projects, version history, and a polished editorial workflow.

What You Will Demonstrate

- Building a Django backend with Django REST Framework (DRF) for API development
- Working with Django's ORM, migrations, and admin panel for data management
- Designing complex data models: projects, content pieces, versions, templates, brand voices
- Advanced prompt engineering for content generation: tone control, audience targeting, SEO optimization
- Implementing structured output: LLM generates content with metadata (titles, tags, word count, readability score)
- Building a production-quality React.js frontend with rich text editing and content management UI
- Batch content generation, content repurposing across formats, and multi-language translation

Why Django?

Django is the most widely used Python web framework in production. Its batteries-included approach (ORM, admin, auth, migrations) makes it ideal for building data-heavy applications. Learning Django alongside Flask gives you versatility - Flask for lightweight APIs, Django for full-featured platforms.

2. Problem Statement

Build an AI Content Generation Platform where users can:

1. Create content projects (e.g., 'Q2 Marketing Campaign', 'Product Launch Blog Series') to organize related content pieces
2. Generate content across 8+ formats: blog posts, social media posts, email campaigns, product descriptions, ad copy, newsletters, landing page copy, press releases
3. Control generation parameters: tone (formal/casual/witty/professional), target audience, word count, language, brand voice, SEO keywords
4. Edit, refine, and iterate on generated content with AI-assisted rewriting
5. Repurpose content across formats (e.g., turn a blog post into 5 social media posts + an email newsletter)
6. Maintain brand voice profiles that ensure consistent tone across all generated content
7. Track content versions, compare drafts, and manage an editorial workflow (Draft -> Review -> Published)

Example Workflow

A marketing manager at a startup:

- Creates a project called 'Product Launch - AI Analytics Tool'
- Sets up a brand voice profile: 'Professional but approachable, data-driven, avoids jargon'
- Generates a 1500-word blog post about the product launch with SEO keywords
- AI produces the blog post with title suggestions, meta description, and tag recommendations
- They refine the intro paragraph using AI rewrite ('make it more compelling')
- One-click repurpose: generates 5 LinkedIn posts, 5 Twitter posts, and 1 email newsletter from the blog
- Translates the email into Spanish and French for international markets
- Exports everything as a content calendar with publish dates

Confidential - For Internal Use Only

3. Technical Requirements

3.1 Content Generation Engine

Build the core AI content generation system with deep customization:

- **Multi-Format Generation:** Support at least 8 content types. Each type has a specialized prompt template, specific output structure, and format-appropriate defaults (e.g., Twitter posts max 280 chars, blog posts 800-2000 words).
- **Tone & Style Control:** Implement at least 8 tone presets (Formal, Casual, Witty, Professional, Friendly, Authoritative, Conversational, Inspirational). Users can also describe custom tones in free text.
- **Audience Targeting:** Allow users to specify target audience (e.g., 'CTOs at mid-size companies', 'College students', 'Non-technical stakeholders'). The LLM adjusts vocabulary, examples, and complexity accordingly.
- **SEO Optimization:** Accept target keywords and generate SEO-optimized content. Output includes: suggested title tags, meta descriptions, header structure (H1/H2/H3), keyword density analysis, internal linking suggestions.
- **Brand Voice Profiles:** Users create reusable brand voice profiles containing: voice description, example phrases, words to use, words to avoid, sample content. These are injected into prompts for consistent output.
- **Structured Output:** Every generation returns: the content itself, a suggested title (3 options), tags/categories, estimated reading time, readability score (Flesch-Kincaid), word count, SEO score.

3.2 Content Types (8 Required + Bonus)

Each content type must have its own specialized generation pipeline:

Content Type	Default Length	Special Features
Blog Post	800-2000 words	SEO structure, H1/H2/H3, meta desc, featured image suggestion
Social Media	Platform-specific	Twitter(280ch), LinkedIn(1300ch), Instagram(2200ch), Facebook
Email Campaign	200-500 words	Subject line(3 options), preview text, CTA, HTML/plain text
Product Description	100-300 words	Features, benefits, specs, comparison points, CTA
Ad Copy	25-90 words	Headlines, descriptions, display URLs (Google Ads format)
Newsletter	500-1000 words	Sections, links, images suggestions, sign-off
Landing Page	300-800 words	Hero section, features, testimonials, CTA, FAQ
Press Release	400-800 words	Headline, dateline, quotes, boilerplate, contact info

Bonus content types: Video scripts, Podcast show notes, Case studies, WhatsApp marketing messages, SMS campaigns.

3.3 Content Editing & Refinement

Build AI-assisted editing capabilities:

- **AI Rewrite:** Select any paragraph or section and ask the AI to rewrite it with instructions (e.g., 'Make shorter', 'Add more data', 'Make more emotional', 'Simplify language').
- **Expand/Compress:** One-click expand (add more detail to a section) or compress (make it shorter while keeping key points).
- **Tone Shift:** Rewrite entire content in a different tone without changing the message (e.g., formal to casual).
- **Grammar & Style Check:** AI-powered grammar check with suggestions. Style consistency check against the brand voice.
- **Rich Text Editor:** Integrate a WYSIWYG editor (Tiptap, Slate.js, or React Quill) for manual editing alongside AI edits.
- **Inline AI Suggestions:** (Bonus) As the user types, show AI autocomplete suggestions (like GitHub Copilot for content).

3.4 Content Repurposing Engine

Transform content from one format to multiple others:

- **One-to-Many:** Take a blog post and generate: 5 social media posts (platform-specific), 1 email newsletter, 3 ad copy variations, 1 summary for newsletter. All in one click.
- **Format Adaptation:** Intelligently adapt content for each format - not just truncation but true reformatting. A LinkedIn post derived from a blog should feel native to LinkedIn.
- **Batch Generation:** Generate multiple variations of the same content type (e.g., 5 different email subject lines, 3 blog title options, 10 social media post variations).
- **Content Calendar:** After batch generation, allow users to arrange content on a calendar view with suggested publish dates and times.

3.5 Translation & Localization

Multi-language content generation:

- **Translation:** Translate any generated content into 10+ languages. Not just word-for-word - culturally adapted translation that preserves tone and intent.
- **Native Generation:** Option to generate content directly in a target language (not translate from English) for more natural results.
- **Multi-Language Variants:** Generate the same content in multiple languages simultaneously. Display side-by-side for review.
- **Localization Notes:** AI provides localization suggestions (e.g., 'This idiom does not translate well to Japanese, consider...').

3.6 Project & Content Management

Build a content management layer:

- **Projects:** Create projects to group related content. Each project has: name, description, brand voice, default settings, team members (future).
- **Content Pieces:** Each piece belongs to a project. Has: content type, status (Draft/Review/Published), version history, tags, metadata.
- **Version History:** Every edit (manual or AI) creates a new version. Users can view diff between versions, restore any previous version.
- **Status Workflow:** Draft -> In Review -> Approved -> Published. Status changes are tracked with timestamps.
- **Search & Filter:** Search content by keyword, filter by project, type, status, date range, tags, language.
- **Dashboard:** Overview showing: total content pieces, generation stats this week/month, content by status, content by type distribution, recent activity.

3.7 Backend API (Django REST Framework)

Build a comprehensive REST API using Django REST Framework:

Method	Endpoint	Description
POST	/api/generate/	Generate content (type, params, brand voice)
POST	/api/generate/batch/	Batch generate (multiple variations)
POST	/api/repurpose/	Repurpose content to other formats
POST	/api/rewrite/	AI rewrite a section with instructions
POST	/api/translate/	Translate content to target language(s)
POST	/api/seo/analyze/	Analyze content for SEO score & suggestions
GET/POST	/api/projects/	List / create projects
GET/PUT/DEL	/api/projects/{id}/	Get / update / delete project
GET/POST	/api/content/	List / create content pieces
GET/PUT/DEL	/api/content/{id}/	Get / update / delete content
GET	/api/content/{id}/versions/	List versions of a content piece
POST	/api/content/{id}/versions/restore/	Restore a previous version
PATCH	/api/content/{id}/status/	Update content status (workflow)
GET/POST	/api/brand-voices/	List / create brand voice profiles
GET/PUT/DEL	/api/brand-voices/{id}/	Manage brand voice profiles
GET/POST	/api/templates/	Content type templates
GET	/api/dashboard/stats/	Dashboard statistics
POST	/api/export/	Export content (Markdown, HTML, PDF, DOCX)
GET	/api/health/	Health check

4. Recommended Tech Stack

Backend (Django)

- **Language:** Python 3.10+
- **Framework:** Django 4.2+ with Django REST Framework (DRF)
- **Database:** PostgreSQL (recommended) or SQLite for development
- **ORM:** Django ORM (built-in) with migrations
- **Serialization:** DRF Serializers for API input/output validation
- **Admin Panel:** Django Admin (register all models for easy data management)
- **LLM Provider:** Google Gemini API (free tier), Groq API (free tier), or OpenAI GPT
- **Background Tasks:** Django-Q2 or Celery with Redis (for batch generation - Good-to-Have)
- **Text Analysis:** textstat (readability scores), nltk (keyword extraction)
- **Export:** python-docx (DOCX export), WeasyPrint or xhtml2pdf (PDF export)

Frontend (React.js)

- **Framework:** React.js 18+ (Create React App or Vite)
- **Routing:** React Router v6
- **State Management:** React Context + useReducer or Redux Toolkit
- **Styling:** Tailwind CSS or Material UI (MUI)
- **Rich Text Editor:** Tiptap (recommended), React Quill, or Slate.js
- **HTTP Client:** Axios with interceptors for error handling
- **Charts:** Recharts or Chart.js (for dashboard)
- **Calendar:** react-big-calendar or FullCalendar (for content calendar view)
- **Markdown:** react-markdown (for preview/rendering)
- **Notifications:** react-hot-toast or react-toastify

DevOps & Tooling

- **Version Control:** Git (mandatory - we will review commit history)
- **Environment:** .env file for API keys, database credentials
- **Docker:** Docker + docker-compose (bonus) with PostgreSQL container
- **Testing:** Django TestCase + DRF APITestCase for backend tests (bonus)

Django Admin is Powerful

Django's admin panel automatically creates a management UI for your database models. Register all your models (Projects, Content, BrandVoice, etc.) in admin.py. This gives you a free, powerful back-office tool for debugging and data management. Reviewers will check your admin setup.

5. Data Models (Django ORM)

Design the following Django models. These form the backbone of your platform. Think carefully about relationships, fields, and constraints.

Project Model

```
class Project(models.Model):
    name = CharField(max_length=200)
    description = TextField(blank=True)
    brand_voice = ForeignKey(BrandVoice, null=True, on_delete=SET_NULL)
    default_tone = CharField(max_length=50, default='professional')
    default_audience = CharField(max_length=200, blank=True)
    default_language = CharField(max_length=10, default='en')
    seo_keywords = JSONField(default=list)  # Default keywords for project
    created_at = DateTimeField(auto_now_add=True)
    updated_at = DateTimeField(auto_now=True)
```

ContentPiece Model

```
class ContentPiece(models.Model):
    STATUS = [('draft','Draft'),('review','In Review'),
              ('approved','Approved'),('published','Published')]
    TYPES = [('blog','Blog Post'),('social','Social Media'),
             ('email','Email'),('product','Product Desc'), ...]

    project = ForeignKey(Project, on_delete=CASCADE, related_name='contents')
    title = CharField(max_length=500)
    content_type = CharField(max_length=20, choices=TYPES)
    body = TextField()  # The actual content (Markdown/HTML)
    status = CharField(max_length=20, choices=STATUS, default='draft')
    tone = CharField(max_length=50)
    target_audience = CharField(max_length=200, blank=True)
    language = CharField(max_length=10, default='en')
    seo_keywords = JSONField(default=list)
    metadata = JSONField(default=dict)  # word_count, readability, seo_score
    tags = JSONField(default=list)
    parent_content = ForeignKey('self', null=True, on_delete=SET_NULL)
    version = IntegerField(default=1)
    created_at = DateTimeField(auto_now_add=True)
    updated_at = DateTimeField(auto_now=True)
```

BrandVoice Model

```
class BrandVoice(models.Model):
    name = CharField(max_length=200)  # e.g., 'Startup Casual'
    description = TextField()  # Detailed voice description
    tone_keywords = JSONField(default=list)  # ['friendly', 'data-driven']
    vocabulary_use = JSONField(default=list)  # Words to USE
```

```
    vocabulary_avoid = JSONField(default=list)  # Words to AVOID
    example_content = TextField(blank=True)  # Sample content in this voice
    is_default = BooleanField(default=False)
    created_at = DateTimeField(auto_now_add=True)
```

ContentVersion Model

```
class ContentVersion(models.Model):
    content_piece = ForeignKey(ContentPiece, on_delete=CASCADE)
    version_number = IntegerField()
    body = TextField()  # Snapshot of content at this version
    change_description = CharField(max_length=500)  # What changed
    change_type = CharField(max_length=20)  # 'ai_generate', 'ai_rewrite', 'manual'
    created_at = DateTimeField(auto_now_add=True)
```

GenerationLog Model

```
class GenerationLog(models.Model):
    content_piece = ForeignKey(ContentPiece, null=True, on_delete=SET_NULL)
    prompt_used = TextField()  # Full prompt sent to LLM
    parameters = JSONField()  # tone, audience, keywords, etc.
    provider = CharField(max_length=50)  # 'gemini', 'openai', 'groq'
    input_tokens = IntegerField()
    output_tokens = IntegerField()
    latency_ms = IntegerField()
    estimated_cost = DecimalField(max_digits=10, decimal_places=6)
    created_at = DateTimeField(auto_now_add=True)
```

6. Expected Project Structure

```
ai-content-platform/
|
|-- backend/
|   |-- manage.py
|   |-- config/
|   |   |-- settings.py            # Django settings
|   |   |-- urls.py                # Root URL config
|   |   |-- wsgi.py
|   |-- content/                   # Main Django app
|   |   |-- models.py              # All data models
|   |   |-- serializers.py         # DRF serializers
|   |   |-- views.py               # API views (ViewSets)
|   |   |-- urls.py                # App URL routes
|   |   |-- admin.py               # Django admin registration
|   |   |-- services/
|   |   |   |-- generator.py       # Content generation logic
|   |   |   |-- repurposer.py      # Content repurposing engine
|   |   |   |-- translator.py      # Translation service
|   |   |   |-- seo_analyzer.py    # SEO analysis & scoring
|   |   |   |-- rewriter.py        # AI rewrite service
|   |   |   |-- llm_client.py      # LLM API integration
|   |   |   |-- text_analyzer.py   # Readability, word count, etc.
|   |   |-- migrations/
|   |   |-- tests/                 # (Bonus) Unit tests
|   |-- requirements.txt
|   |-- Dockerfile
|
|-- frontend/
|   |-- src/
|   |   |-- components/
|   |   |   |-- Dashboard.jsx      # Stats & overview
|   |   |   |-- ProjectList.jsx    # Project management
|   |   |   |-- ContentEditor.jsx  # Rich text editor + AI tools
|   |   |   |-- GenerationForm.jsx # Content generation wizard
|   |   |   |-- RepurposePanel.jsx # One-to-many repurposing
|   |   |   |-- BrandVoiceForm.jsx # Brand voice editor
|   |   |   |-- ContentList.jsx    # Content listing with filters
|   |   |   |-- VersionHistory.jsx # Version comparison view
|   |   |   |-- ContentCalendar.jsx # Calendar view
|   |   |   |-- SEOPanel.jsx       # SEO analysis display
|   |   |-- pages/
|   |   |-- hooks/
|   |   |-- services/api.js        # API client
|   |   |-- App.jsx
|   |-- package.json
|   |-- Dockerfile
|
```

```
|-- docker-compose.yml              # (Bonus)
|-- .env.example
|-- .gitignore
|-- README.md
```

7. UI Screens & Layout

Screen 1: Dashboard

- Total content pieces (by status: draft, review, published)
- Content generated this week/month (bar chart)
- Content by type distribution (pie chart)
- Recent activity feed (latest 10 generated/edited pieces)
- Quick-generate buttons for common content types
- Top-performing brand voice (most used)

Screen 2: Projects

- Grid of project cards with: name, description, content count, brand voice, last updated
- Create new project modal/page with all fields
- Click project to enter project workspace showing all its content pieces
- Project settings (default tone, audience, keywords, brand voice)

Screen 3: Content Generation Wizard

- Step 1: Choose content type (8 types as visual cards with icons and descriptions)
- Step 2: Configure parameters - tone, audience, word count, language, SEO keywords, brand voice
- Step 3: Provide context/brief - what the content should be about (text area with AI suggestions)
- Step 4: Generate & Review - show AI output with metadata (readability, word count, SEO score)
- Step 5: Edit in rich text editor, save to project, or regenerate

Screen 4: Content Editor

- Full-page rich text editor (Tiptap/Quill) with formatting toolbar
- Right sidebar: AI tools panel - Rewrite, Expand, Compress, Tone Shift, Grammar Check
- Select text -> right-click or toolbar button -> AI rewrite with instructions
- SEO panel showing: keyword density, readability score, meta description preview, suggestions
- Version history panel: list of all versions with 'Restore' button
- Status dropdown: Draft -> In Review -> Approved -> Published
- Export dropdown: Markdown, HTML, PDF, DOCX, Copy to clipboard

Screen 5: Repurpose View

- Source content displayed on the left
- Target format checkboxes on the right (select which formats to generate)
- 'Repurpose' button generates all selected formats
- Results displayed in tabs or cards - one per generated format

- Edit any result, then save all to project

Screen 6: Brand Voice Manager

- List of all brand voice profiles with preview
- Create/edit form with: name, description, tone keywords, use/avoid words, example content
- 'Test Voice' button: enter sample text, see how AI would rewrite it in this voice
- Set default voice for a project

Screen 7: Content Calendar (Good-to-Have)

- Monthly/weekly calendar view
- Drag content pieces onto calendar dates
- Color-coded by content type and status
- Export calendar as CSV/iCal

8. Feature Breakdown & Priority

Must-Have (Core - Required)

1. Django backend with DRF, proper models, serializers, ViewSets, and URL routing
2. Django Admin registered for all models (Project, ContentPiece, BrandVoice, etc.)
3. Content generation for at least 6 content types with specialized prompts per type
4. Tone control (at least 6 presets), audience targeting, and word count configuration
5. Brand voice profiles: create, edit, delete, apply to generation
6. Content projects: CRUD, with content pieces organized under projects
7. Rich text editor for post-generation editing (Tiptap or React Quill)
8. AI rewrite: select text and rewrite with instructions (make shorter, change tone, etc.)
9. Content repurposing: convert one content type to at least 3 other formats in one click
10. Version history: every edit tracked, view previous versions, restore any version
11. Status workflow: Draft -> Review -> Approved -> Published
12. SEO analysis: keyword density, readability score, meta description, title tag suggestions
13. Dashboard with stats: total content, content by type, content by status, recent activity
14. React.js frontend with routing, clean responsive UI, proper state management
15. Generation metadata: word count, readability score, token usage, response time

Good-to-Have (Intermediate)

1. Translation to 5+ languages with localization notes
2. Batch generation: generate 5-10 variations of the same content type
3. Content calendar view with drag-and-drop scheduling
4. Content export: Markdown, HTML, PDF, DOCX formats
5. Expand/Compress AI tools for any content section
6. Grammar and style check with AI suggestions
7. Search and filter across all content (by keyword, type, status, date, tags)
8. Generation log tracking all LLM calls with tokens, cost, latency

Bonus (Advanced)

1. Inline AI autocomplete suggestions while typing in the editor
2. Streaming content generation (tokens appear as they are generated)
3. A/B content testing: generate 2 versions, compare side-by-side, rate them
4. Content scoring AI: rate generated content on creativity, engagement, clarity (1-10)
5. Docker + docker-compose with PostgreSQL container
6. Django unit tests for key API endpoints (at least 10 tests)

Confidential - For Internal Use Only

7. Plagiarism-style check: ensure generated content is sufficiently unique across your library
8. Multi-provider support: generate with different LLMs and compare quality
9. Content analytics: which tones/audiences produce the highest-rated content
10. Webhook/API for publishing: push content to WordPress, Medium, or Mailchimp (mock or real)

9. Evaluation Criteria

Criteria	Weight	What We Look For
Functionality	25%	All content types work, repurposing, editing, versioning, workflow
Django Proficiency	20%	Proper models, DRF usage, admin setup, migrations, serializers
GenAI Understanding	20%	Quality of generation prompts, tone control, brand voice, SEO
UI/UX	15%	Clean dashboard, smooth editor, intuitive wizard, responsive design
Code Quality	10%	Clean structure, DRY code, proper error handling, modularity
Documentation	5%	README, API docs, screenshots, Django admin setup
Bonus Features	5%	Docker, tests, streaming, calendar, export, analytics

What Impresses Us

- Content that reads like a human wrote it - not generic AI slop. Prompts must be well-crafted per content type.
- Brand voice profiles that genuinely change the output style - not just a label
- Repurposing that produces format-native content (LinkedIn post feels like LinkedIn, not truncated blog)
- Clean Django code: proper use of ViewSets, serializer validation, queryset optimization
- Rich text editor that integrates AI tools smoothly (select -> rewrite feels seamless)
- Version history with actual diffs showing what changed between versions
- Dashboard that tells a useful story with real data visualization

Common Mistakes to Avoid

- Using the same generic prompt for all content types (each type needs specialized prompting)
- Not using Django REST Framework properly (raw views instead of ViewSets/serializers)
- Skipping Django Admin registration (it is free, powerful, and we check for it)
- No version tracking - losing content history after edits
- Repurposing that is just truncation/copy-paste instead of intelligent format adaptation
- Frontend with no routing - everything on one page instead of proper multi-page navigation
- No loading indicators during content generation (LLM calls take time)
- Not validating API inputs with DRF serializers (accepting any garbage data)

10. Submission Guidelines

What to Submit

1. GitHub Repository - Full commit history, clean .gitignore, no database files or .env committed.
2. README.md - Setup instructions (Django + React), architecture overview, screenshots of all screens, API

documentation (or link to DRF browsable API), list of content types supported.

3. .env.example - All required environment variables documented.
4. Demo Video (Recommended) - 5-7 min walkthrough: create project, generate content, edit with AI, repurpose, show dashboard, demonstrate brand voice.

Submission Deadline

Submit within 4 days of receiving this assignment. Communicate early if you need more time.

11. Getting Started Guide

Day 1: Django Backend Foundation

- Initialize Django project: django-admin startproject config . && python manage.py startapp content
- Define all data models (Project, ContentPiece, BrandVoice, ContentVersion, GenerationLog)
- Run migrations: python manage.py makemigrations && python manage.py migrate
- Register all models in admin.py with custom admin views (list_display, filters, search)
- Install Django REST Framework, create serializers for all models
- Create ViewSets and URL routing for all API endpoints
- Test all CRUD endpoints via DRF browsable API or Postman

Day 2: Content Generation Engine

- Integrate LLM API (Gemini/Groq/OpenAI) in llm_client.py
- Build the content generator service with specialized prompts per content type
- Implement tone control, audience targeting, and brand voice injection into prompts
- Build the repurposing engine (blog -> social + email + ad)
- Implement AI rewrite service (rewrite with instructions)
- Add SEO analysis (keyword density, readability scoring via textstat)
- Implement version tracking on every content edit

Day 3: React.js Frontend

- Initialize React project with Vite + Tailwind CSS + React Router
- Build dashboard page with charts (Recharts)
- Build project list and project detail pages
- Build content generation wizard (multi-step form)
- Integrate rich text editor (Tiptap recommended) with AI rewrite toolbar
- Build content list with search, filter, and status management
- Build brand voice manager page

Day 4: Integration, Polish & Submit

- Build repurpose view: select source content, choose target formats, generate all
- Implement version history view with restore functionality
- Add export functionality (Markdown, HTML at minimum)
- Test all content types with various tones, audiences, and brand voices
- Add error handling, loading states, validation throughout
- Write comprehensive README with screenshots
- Record demo video and submit

12. Helpful Resources

Django & DRF

- Django Documentation: docs.djangoproject.com
- Django REST Framework: django-rest-framework.org
- DRF ViewSets Tutorial: django-rest-framework.org/api-guide/viewsets/
- Django Admin Customization: docs.djangoproject.com/en/4.2/ref/contrib/admin/

React.js & Frontend

- React Documentation: react.dev
- React Router: reactrouter.com
- Tiptap Editor: tiptap.dev (recommended rich text editor)
- Recharts: recharts.org (charts for dashboard)
- Tailwind CSS: tailwindcss.com

Content & SEO

- textstat (Python): pypi.org/project/textstat/ (readability scoring)
- SEO best practices: search 'on-page SEO checklist 2024'
- Content marketing formats: search 'content types for marketing'

LLM APIs

- Google Gemini API: ai.google.dev (free tier)
- Groq API: console.groq.com (fast, free tier)
- OpenAI API: platform.openai.com/docs

13. Questions?

This assignment tests both your Django skills and your ability to build a product that real users would want. The content quality matters - prompts must be carefully crafted so that each content type produces genuinely useful output. If you have questions about requirements, reach out.

Good luck! Build a platform that makes content creation feel effortless.

--- End of Assignment #4 ---